

SOURCE CODE

Import required libraries

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import LabelEncoder
from google.colab import files
```

Upload CSV file

```
uploaded = files.upload()
filename = list(uploaded.keys())[0]
```

Load dataset

```
df = pd.read_csv(filename)
```

Encode movie names for matrix creation

```
movie_encoder = LabelEncoder()
df['movie_encoded'] = movie_encoder.fit_transform(df['movie'])
```

Create user-item matrix

```
user_item_matrix = df.pivot_table(index='userId', columns='movie_encoded', values='rating').fillna(0)
```

Compute similarity between users

```
user_similarity = cosine_similarity(user_item_matrix)
```

Convert to DataFrame

```
similarity_df = pd.DataFrame(user_similarity, index=user_item_matrix.index,
                             columns=user_item_matrix.index)
```

```
def get_recommendations(user_id, top_n=3):
```

 Get similarity scores for all users except the target user

```
    similar_users = similarity_df[user_id].drop(user_id).sort_values(ascending=False)
```

 Filter the user-item matrix to include only similar users

```
    similar_users_matrix = user_item_matrix.loc[similar_users.index]
```

 Calculate weighted scores using the filtered matrix

```
    weighted_scores = similar_users_matrix.T.dot(similar_users) / similar_users.sum()
```

```
    user Rated movies = user_item_matrix.loc[user_id]
```

```
    recommendations = weighted_scores[user Rated movies ==
0].sort_values(ascending=False).head(top_n)
```

```
    recommended_movie_ids = recommendations.index
```

```
    recommended_movies = movie_encoder.inverse_transform(recommended_movie_ids)
```

```
    return list(recommended_movies)
```